K means Clustering – Introduction

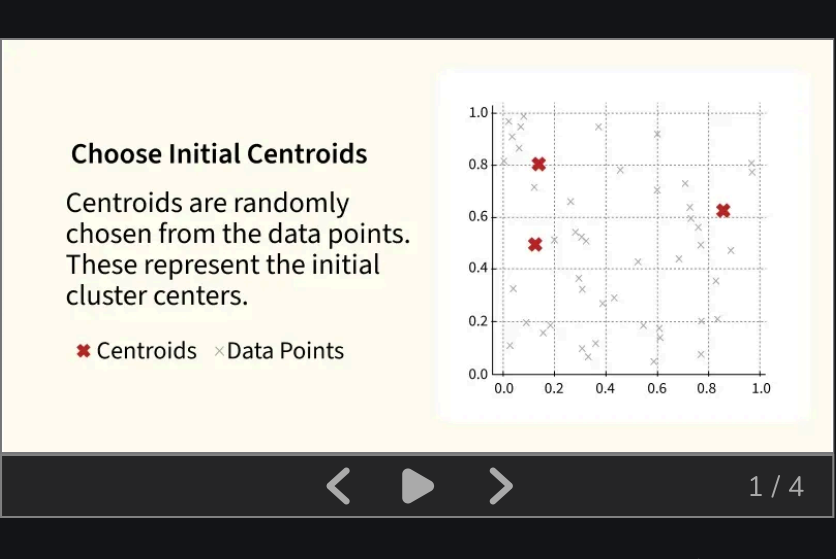
Last Updated : 1 May, 2026

K-Means Clustering groups similar data points into clusters without needing labeled data. It is used to uncover hidden patterns when the goal is to organize data based on similarity.

- Helps identify natural groupings in unlabeled datasets
- Works by grouping points based on distance to cluster centers
- Commonly used in customer segmentation, image compression and pattern discovery
- Useful when you need structure from raw, unorganized data

Working of K-Means Clustering

Suppose we are given a data set of items with certain features and values for these features like a vector. The task is to categorize those items into groups. To achieve this we will use the K-means algorithm. "k" represents the number of groups or clusters we want to classify our items into.



The algorithm will categorize the items into "k" groups or clusters of similarity. To calculate that similarity we will use the Euclidean distance as a measurement. The algorithm works as follows:

1. **Initialization:** We begin by randomly selecting k cluster centroids.
2. **Assignment Step:** Each data point is assigned to the nearest centroid, forming clusters.
3. **Update Step:** After the assignment, we recalculate the centroid of each cluster by averaging the points within it.
4. **Repeat:** This process repeats until the centroids no longer change or the maximum number of iterations is reached.

The goal is to partition the dataset into k clusters such that data points within each cluster are more similar to each other than to those in other clusters.

> *Selecting the right number of clusters is important for meaningful segmentation to do this we use Elbow Method for optimal value of k in KMeans which is a graphical tool used to determine the optimal number of clusters (k) in K-means.*

Uses of K-Means Clustering

K-Means is popular in a wide variety of applications due to its simplicity, efficiency and effectiveness. Here's why it is widely used:

1. **Data Segmentation:** One of the most common uses of K-Means is segmenting data into distinct groups. For example, businesses use K-Means to group customers based on behavior, such as purchasing patterns or website interaction.
2. **Image Compression**: K-Means can be used to reduce the complexity of images by grouping similar pixels into clusters, effectively compressing the image. This is useful for image storage and processing.
3. **Anomaly Detection:** K-Means can be applied to detect anomalies or outliers by identifying data points that do not belong to any of the clusters.
4. **Document Clustering:** In natural language processing (NLP), K-Means is used to group similar documents or articles together. It's often used in applications like recommendation systems or news categorization.
5. **Organizing Large Datasets:** When dealing with large datasets, K-Means can help in organizing the data into smaller, more manageable chunks based on similarities, improving the efficiency of data analysis.

Implementation of K-Means Clustering

We will be using blobs datasets and show how clusters are made using Python programming language.

Step 1: Importing the necessary libraries

We will be importing the following libraries.

- Numpy**:** for numerical operations (e.g., distance calculation).
- Matplotlib: for plotting data and results.
- Scikit learn**:** to create a synthetic dataset using **make_blobs**

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
```

Step 2: Creating Custom Dataset

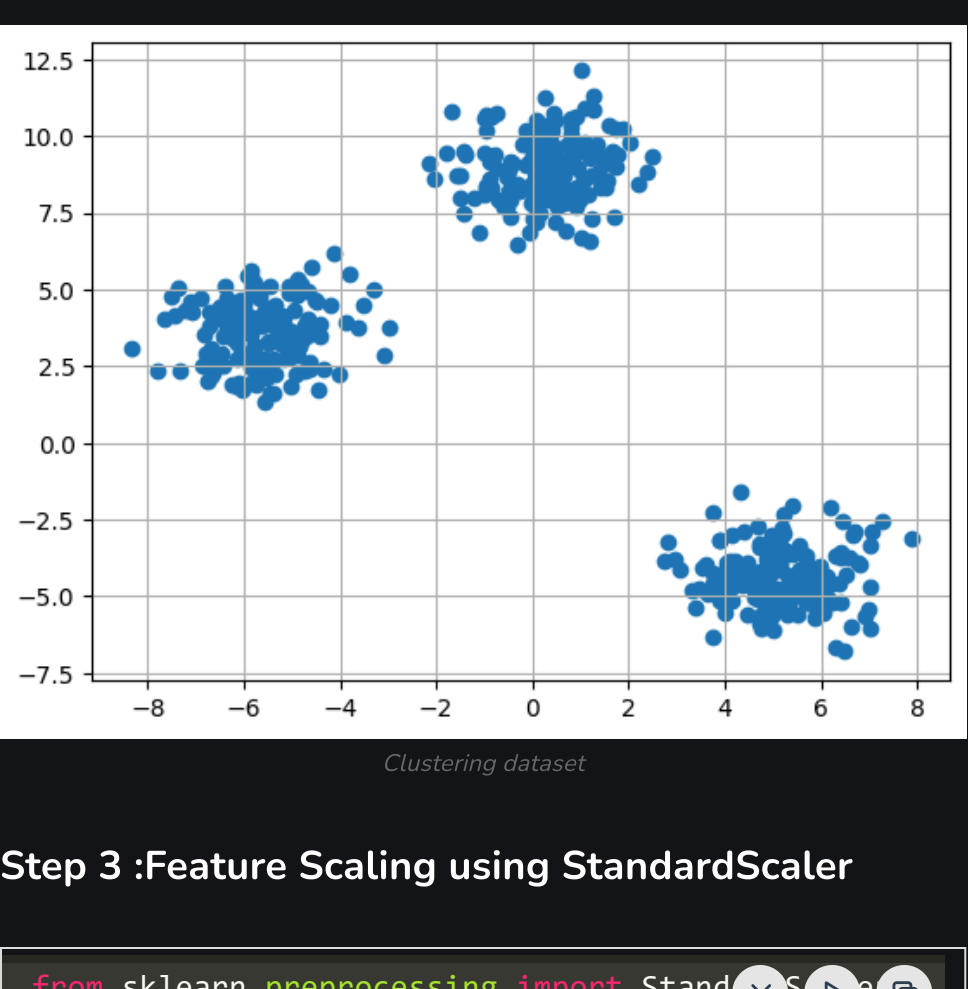
We will generate a synthetic dataset with make_blobs.

- **make_blobs(n_samples=500, n_features=2, centers=3):** Generates 500 data points in a 2D space, grouped into 3 clusters.
- **plt.scatter(X[:, 0], X[:, 1]):** Plots the dataset in 2D, showing all the points.
- **plt.show():** Displays the plot

```
X,y = make_blobs(n_samples = 500,n_features = 2,centers = 3,random_state = 23)

fig = plt.figure(0)
plt.grid(True)
plt.scatter(X[:,0],X[:,1])
plt.show()
```

Output:



Clustering dataset

Step 3 :Feature Scaling using StandardScaler

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X = scaler.fit_transform(X)
```

Step 4: Initializing Random Centroids

We will randomly initialize the centroids for K-Means clustering

- **np.random.seed(23):** Ensures reproducibility by fixing the random seed.
- The for loop initializes k random centroids, with values between -2 and 2, for a 2D dataset.

```
k = 3

clusters = {}
np.random.seed(23)

for idx in range(k):
    center = 2*(2*np.random.random((X.shape[1],))-1)
    points = []
    cluster = {
        'center' : center,
        'points' : []
    }

    clusters[idx] = cluster

clusters
```

Output:

```
{0: {'center': array([0.06919154, 1.78785042]), 'points': []},
 1: {'center': array([ 1.06183904, -0.87041662]), 'points': []},
 2: {'center': array([-1.11581855,  0.74488834]), 'points': []}}
```

Random Centroids

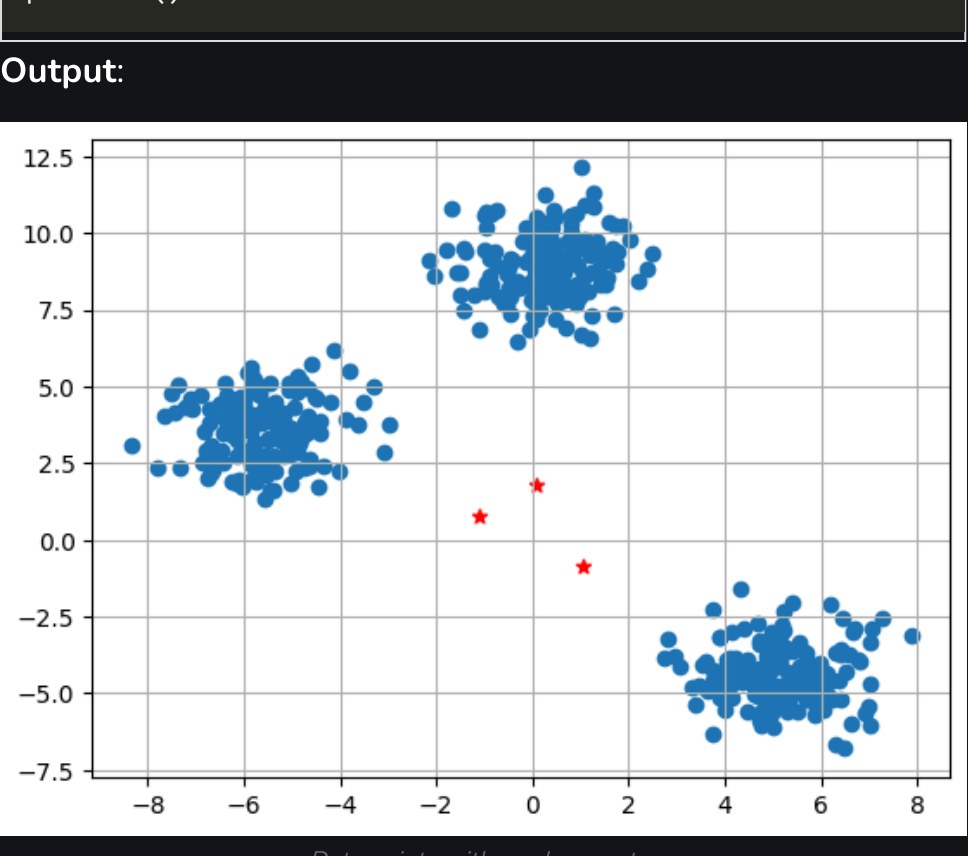
Step 5: Plotting Random Initialized Center with Data Points

We will now plot the data points and the initial centroids.

- **plt.grid()**: Plots a grid.
- **plt.scatter(center[0], center[1], marker='*', c='red'):** Plots the cluster center as a red star (* marker).

```
plt.scatter(X[:,0],X[:,1])
plt.grid(True)
for i in clusters:
    center = clusters[i]['center']
    plt.scatter(center[0],center[1],marker = '*',c = 'red')
plt.show()
```

Output:



Data points with random center

Step 6: Defining Euclidean Distance

To assign data points to the nearest centroid, we define a distance function:

- **np.sqrt():** Computes the square root of a number or array element-wise.
- **np.sum():** Sums all elements in an array or along a specified axis

```
def distance(p1,p2):
    return np.sqrt(np.sum((p1-p2)**2))
```

Step 7: Creating Assign and Update Functions

Next, we define functions to assign points to the nearest centroid and update the centroids based on the average of the points assigned to each cluster.

- **dist.append(dis):** Appends the calculated distance to the list dist.
- **curr_cluster = np.argmin(dist):** Finds the index of the closest cluster by selecting the minimum distance.
- **new_center = points.mean(axis=0):** Calculates the new centroid by taking the mean of the points in the cluster.

```
def assign_clusters(X, clusters):
    for idx in range(X.shape[0]):
        dist = []

        curr_x = X[idx]

        for i in range(k):
            dis = distance(curr_x,clusters[i]['center'])
            dist.append(dis)
        curr_cluster = np.argmin(dist)
        clusters[curr_cluster]['points'].append(curr_x)
    return clusters

def update_clusters(X, clusters):
    for i in range(k):
        points = np.array(clusters[i]['points'])
        if points.shape[0] > 0:
            new_center = points.mean(axis =0)
            clusters[i]['center'] = new_center

            clusters[i]['points'] = []
    return clusters
```

Step 8: Predicting the Cluster for the Data Points

We create a function to predict the cluster for each data point based on the final centroids.

- **pred.append(np.argmin(dist)):** Appends the index of the closest cluster (the one with the minimum distance) to pred.

```
def pred_cluster(X, clusters):
    pred = []
    for i in range(X.shape[0]):
        dist = []
        for j in range(k):
            dist.append(distance(X[i],clusters[j]['center']))
        pred.append(np.argmin(dist))
    return pred
```

Step 9: Assigning, Updating and Predicting the Cluster Centers

The assign and update steps are repeated multiple times until the cluster centers stabilize or a maximum number of iterations is reached.

- **assign_clusters(X, clusters):** Assigns data points to the nearest centroids.
- **update_clusters(X, clusters):** Recalculates the centroids.
- **pred_cluster(X, clusters):** Predicts the final clusters for all data points.

```
clusters = assign_clusters(X,clusters)
clusters = update_clusters(X,clusters)
pred = pred_cluster(X,clusters)
```

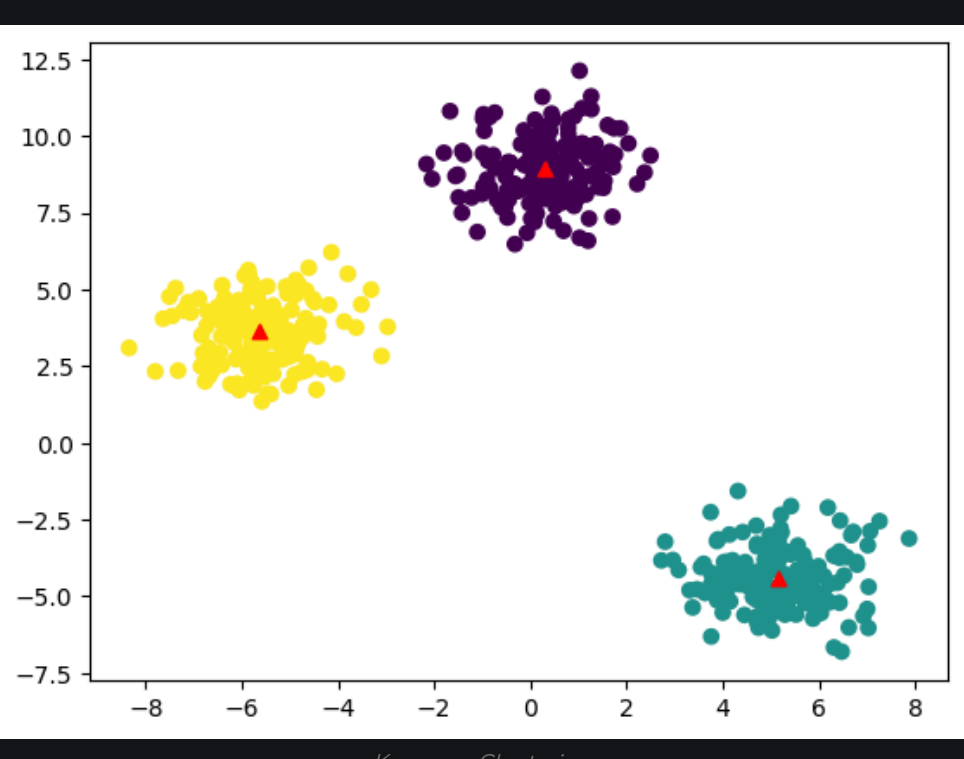
Step 10: Plotting Data Points with Predicted Cluster Centers

Finally, we plot the data points, colored by their predicted clusters, along with the updated centroids.

- **center = clusters[i]['center']:** Retrieves the center (centroid) of the current cluster.
- **plt.scatter(center[0], center[1], marker='^', c='red'):** Plots the cluster center as a red triangle (^ marker).

```
plt.scatter(X[:,0],X[:,1],c = pred)
for i in clusters:
    center = clusters[i]['center']
    plt.scatter(center[0],center[1],marker = '^',c = 'red')
plt.show()
```

Output:



K-means Clustering

> *You can download the source code from here.*

Challenges with K-Means Clustering

- **Choosing the Right Number of Clusters** (k): One of the biggest challenges is deciding how many clusters to use.
- **Sensitive to Initial Centroids:** The final clusters can vary depending on the initial random placement of centroids.
- **Non-Spherical Clusters:** K-Means assumes that the clusters are spherical and equally sized. This can be a problem when the actual clusters in the data are of different shapes or densities.
- **Outliers**: K-Means is sensitive to outliers, which can distort the centroid and, ultimately, the clusters.